

Home Assignment 2026 – odpovídání



Varování: Znění testových otázek je autorským dílem. Šíření otázek bez písemného souhlasu autora je porušením autorských práv a jako takové může být postihováno dle platných zákonů.

DB Performance Engineer Challenge

You were hired to optimize a growing e-commerce PostgreSQL database. The company reports slow analytics, slow customer search, and delayed order processing. You must investigate the workload and propose improvements.

This assignment focuses on:

- reading execution plans
- planner behavior
- workload-oriented indexing
- tradeoffs in optimization
- recognizing when indexes do not help

Data to work with

The contents of tables are *randomly generated for each student*. Download the [data](#) and follow the procedure below. Note: Download is accessible from FI network only, so use [VPN](#)).

Schema of tables:

- `customers(customer_id, is_active, first_name, middle_name, last_name, email, phone, ...)` -- all customers of our e-shop

- `products(product_id, name, description, price, stock_quantity)` -- products that we trade with
- `orders(order_id, customer_id, order_date, total_amount, status)` -- all (also history) orders placed by our customers
- `ordered_items(item_id, order_id, product_id, quantity, unit_price, vat_perc)` -- items that constitutes contents of individual orders.

Execution Environment

In this task, you will install PostgreSQL on your HW, load the data provided to you and perform DB admin tasks.

To access the PostgreSQL server, you may use `psql` (a command line tool), [PgAdmin](#) (a web-based locally-installed tool), or another PostgreSQL client.

Manual setup

Run the following steps as a DB admin user (postgres):

- Create the database named `pa152_db`
- Populate your own Pg DB instance, e.g., `psql -p 5432 -h localhost -d pa152_db -f <UC0>.sql`
- There is an account "user" with the password "pa152pwd" created in .sql, so you may use it from now on.

Docker container

You may use this [setup package](#) to create a Docker container and run queries.

Runtime Measurement Methodology

Execution time from a single ``EXPLAIN ANALYZE`` run may be unstable because of cache warmup, background activity, checkpoints, and OS scheduling. Therefore:

- use `EXPLAIN (ANALYZE, BUFFERS)` for **plan inspection**,
- use `pgbench` for **runtime measurement**.

For every query whose runtime is to be reported:

1. execute the query once with `EXPLAIN (ANALYZE, BUFFERS)` and use that output for plan analysis,
2. benchmark the same query with `pgbench`,
3. perform one warm-up run of 20 executions,
4. perform one measured run of 50 executions,
5. report the **average latency** from the measured run in milliseconds.

Use exactly one client and one thread:

```
pgbench -n -f QUERY.sql -t 20 -c 1 -j 1 -U user pa152_db
```

```
pgbench -n -f QUERY.sql -t 50 -c 1 -j 1 -U user pa152_db
```

If PostgreSQL is running in Docker, you may run `pgbench` inside the container. Example:

```
docker cp QUERY.sql pa152:/tmp/QUERY.sql
```

```
docker exec -e PGPASSWORD=pa152pwd -it pa152 pgbench -n -
```

```
docker exec -e PGPASSWORD=pa152pwd -it pa152 pgbench -n -
```

◀  ▶

Report the value shown as: `latency average = ... ms`

1. Query Diagnostics Under Real Workload (3 points)

The e-shop reports slow analytical queries. Your task is to identify the main bottlenecks. Run the following queries and inspect their execution plans.

Query A

```
SELECT customer_id, COUNT(*) AS orders_count
FROM orders
GROUP BY customer_id
ORDER BY orders_count DESC
LIMIT 20;
```

Paste full explain plan:

Query B

```
SELECT *
FROM orders
WHERE status = 'Pending'
      AND order_date >= (
        SELECT MAX(order_date) - INTERVAL '30 days'
        FROM orders
      );
```

Paste full explain plan:

Query C

```
SELECT p.product_id, p.name, SUM(oi.quantity) AS sold_qty
FROM ordered_items oi
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name
ORDER BY sold_qty DESC
LIMIT 10;
```

Paste full explain plan:

Which query had the longest execution time? (A/B/C)

Which query read the most shared buffers? (A/B/C)

Which query had the largest estimation error? (A/B/C)

Dominant expensive operator of the slowest query: (Example: Seq Scan, Hash Join, Sort, Aggregate, Nested Loop). It is not Seq scan on table !

Explain in max. 60 words why the query was slow.

2. Index Budget Challenge (4 points)

You may create **at most TWO indexes** on table `orders`.

Your goal is to minimize the **sum of average latencies** of the following workload.

Query Q1

```
SELECT order_id, customer_id, order_date, total_amount, status
FROM orders
WHERE customer_id = 4832
      AND status = 'Pending'
ORDER BY order_date DESC
LIMIT 20;
```

Query Q2

```
SELECT order_id, customer_id, order_date
FROM orders
WHERE status = 'Pending'
      AND order_date >= '2026-04-01';
```

Query Q3

```
SELECT order_id, customer_id, total_amount, order_date
FROM orders
WHERE customer_id = 4832
      AND order_date >= '2026-04-01'
ORDER BY order_date DESC
LIMIT 50;
```

Rules:

- At most two indexes.
- Only on table `orders`.
- Composite indexes are allowed.
- Queries must remain unchanged.
- No helper tables, materialized views, or query rewriting.
- Existing primary-key and foreign-key constraints must remain unchanged.

Average latency of Q1 before indexing (ms)

Average latency of Q2 before indexing (ms)

Average latency of Q3 before indexing (ms)

SQL commands creating your indexes:

Average latency of Q1 after indexing (ms)

Average latency of Q2 after indexing (ms)

Average latency of Q3 after indexing (ms)

Improvement percentage $(\text{before_sum} - \text{after_sum}) / \text{before_sum} * 100$

Explain in max. 80 words which query benefited most, which least, and why your chosen indexes are a compromise.

3. A Plausible but Weak Index (3 points)

Some indexes look useful but bring almost no benefit.

Consider the following query:

```
SELECT order_id, customer_id, order_date, total_amount
FROM orders
WHERE status = 'Shipped';
```

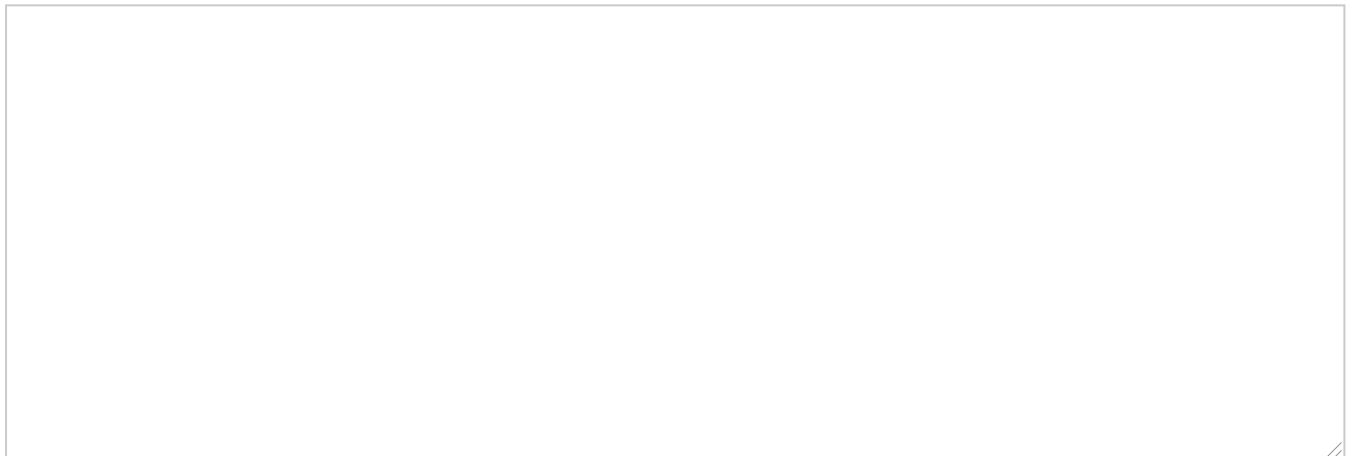
Create **one ordinary B-tree index** on `orders` that looks reasonable for this query, but improves runtime marginally (e.g., less than 10% on your dataset).

Compare the execution plans before and after index creation using `EXPLAIN (ANALYZE, BUFFERS)` `...`.

Rules:

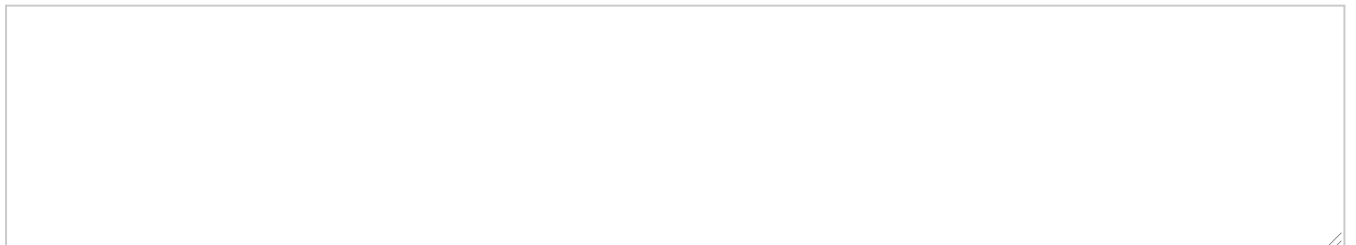
- Exactly one new index.
- Ordinary B-tree only.
- No partial indexes.
- No `INCLUDE` indexes.
- No expression indexes.
- Query must remain unchanged.
- Do not disable planner features.
- Do not modify PostgreSQL configuration.
- Do not delete or update data.

Execution plan before indexing:

A large, empty rectangular box with a thin grey border, intended for the execution plan before indexing. A small double-slash icon is in the bottom right corner.

Average latency before indexing (ms)

SQL command creating the index:

A large, empty rectangular box with a thin grey border, intended for the SQL command creating the index. A small double-slash icon is in the bottom right corner.

Execution plan after indexing:

Average latency after indexing (ms)

Access method after indexing: (example: Seq Scan, Index Scan, Bitmap Heap Scan, Other)

Explain in max. 80 words why the index helped little:

J. Prokop, učo 525248, 5. 5. 2026 14:17.25